Module 4 — Orchestration: the full flow in one place

Build the entire A → B → C → Google Sheet pipeline on ONE canvas, from scratch.

This is a standalone, self-contained runbook. It does **not** use copy-paste from the single-agent workflows — you build every node here, in order, so the whole flow lives in one place. Everything you need (prompts, JSON, expressions, column mapping) is repeated inline below.

> Companion to the main manual. The individual-agent sections there are still useful for teaching each agent alone; this file is the clean end-to-end build for Module 4.

Agents as a team

What you'll build: `Chat → Agent A (Failure Prediction) → Agent B (Spare Parts) → Agent C (Maintenance) → Google Sheets (Append Row)`

Before you start (2 things ready)

- A **Gemini API key** copied (`aistudio.google.com` → Get API key). You paste it once; all three agents reuse it.
- Your **n8n** workflows home open (`*.app.n8n.cloud`).
- A **stable Flash model** in mind (e.g. `gemini-3.6-flash`) — not a "preview" one (preview = ~20 requests/day; stable = ~1,500).

Step 1 — New workflow + chat trigger

1. Workflows home → **Create Workflow** → rename it **"Module 4 — Orchestration (A→B→C→Sheet)"**.
2. Click **+** (Add first step) → **When chat message received** (Chat Trigger).

Step 2 — Agent A (Failure Prediction)

1. From the Chat Trigger, click **+** → search **AI Agent** → add it. **Rename the node to `Agent A`** (double-click its title).
2. Under the AI Agent's **Chat Model** slot → **+** → **Google Gemini Chat Model**.
    - **Credential to connect** → **Create new** → paste your Gemini API key → Save.
    - **Model** → pick your stable Flash model.
3. Double-click **Agent A** → **Prompt** = **"Take from previous node automatically"** (it reads the chat question).
4. Scroll to **Options** → **Add Option** → **System Message** → paste:

```
You are the Failure Prediction Agent for IM-500 injection molding (Bay 3). Predict
failures from sensor readings using ONLY this reference. If not covered, say so -
don't guess.

Thresholds (Normal / Warning / Critical -> cause, lead time):
- Vibration mm/s: <2.8 / 2.8-4.5 / >4.5 -> pump bearing wear, ~7d
- Oil temp C: <50 / 50-60 / >60 -> pump/cooling wear, ~10d
- Heater drift C: +/-5 / 5-10 / >10 -> heater band ageing, ~14d
- Motor current A: 18-24 / 24-28 / >28 -> overload, ~5d
- Screw torque %: <70 / 70-85 / >85 -> screw/barrel wear, ~12d
- Tie-bar strain uE: <800 / 800-1000 / >1000 -> tie-bar wear, ~9d
- Servo error counts: <50 / 50-120 / >120 -> encoder EN-77, ~6d

Rules: Warning = predicted failure; Critical = immediate. Schedule maintenance
before (today + lead time).
Output: failing component, lead time, urgency.
```

1. Wire: **Chat Trigger** → **Agent A** (drag from the trigger's right dot to Agent A's left dot).

Step 3 — Agent B (Spare Parts)

1. From **Agent A**, click **+** → **AI Agent**. **Rename it to `Agent B`.**
2. Under its **Chat Model** slot → **+** → **Google Gemini Chat Model** → pick the **same credential** from the dropdown (don't create a new one) → same model.
3. Double-click **Agent B** → **Prompt** = **"Define below"** → toggle the field to **Expression** → enter:

```
{{ $json.output }}
```

(That's Agent A's output — the predicted failure — flowing in.)

1. **Options** → **Add Option** → **System Message** → paste:

```
You are the Spare Parts Agent for IM-500 (Bay-3 store). Given a failing component/
part, return the exact part + stock using ONLY this reference. Don't invent parts
or stock.

Component -> Part No (stock / min / lead / status):
```

```
- pump bearing -> BR-3105 (4/2/5d OK)
- pump seal -> SK-4620 (1/2/7d LOW)
- barrel heater -> HB-220 (3/2/6d OK)
- thermocouple -> TC-11 (0/1/4d OUT)
- cooling circuit -> CF-08 (6/3/3d OK)
- injection screw -> ST-90 (0/1/15d OUT)
- ejector -> EP-12 (2/1/5d OK)
- servo drive fan -> FN-40 (1/1/9d OK)
- servo encoder -> EN-77 (0/1/21d OUT)

Rules: stock < min -> reorder. If failure sooner than lead time -> URGENT/expedite.
EN-77 always URGENT (21d lead vs ~6d failure), escalate to Bay-3 lead.
Output: part no, stock, lead time, reorder/urgent decision.
```

1. Wire: **Agent A** → **Agent B**.

Step 4 — Agent C (Maintenance) + structured output

1. From **Agent B**, click **+** → **AI Agent**. **Rename it to `Agent C`.**
2. Under its **Chat Model** slot → **+** → **Google Gemini Chat Model** → same credential, same model.
3. Double-click **Agent C** → **Prompt** = **"Define below"** → toggle to **Expression** → enter (this pulls from BOTH A and B):

```
Predicted failure: {{ $('Agent A').first().json.output }}
Parts status: {{ $('Agent B').first().json.output }}
```

> Use `.first()`, not `.item` — Agent A is two nodes back, and `.item` returns blank across two hops. If the field errors, use the single-expression form:
> `{{ "Predicted failure: " + $('Agent A').first().json.output + "\nParts status: " + $('Agent B').first().json.output }}`

1. **Options** → **Add Option** → **System Message** → paste:

```
You are the Maintenance Agent for IM-500 injection molding (Bay 3). You receive a
predicted failure and its spare-part status. Raise a maintenance request using ONLY
that input - don't invent details.

Keep two numbers separate: FAILURE lead time (days to failure) vs PART supply lead
time (days to get the part).
Rules:
- schedule_by_days must be LESS than failure_days (schedule before failure). Never
use the part supply lead time as the schedule date.
- If part out of stock OR part supply lead time > failure lead time -> priority
URGENT, note escalation in summary.
- Otherwise NORMAL.

Output these fields:
- machine: "IM-500 (Bay 3)"
- component: EXACTLY one of: Servo Encoder EN-77, Hydraulic Pump Bearing, Injection
```

```
Screw, Barrel Heater Band, Cooling Circuit, Hydraulic Pump Seal, Thermocouple
- part: part code ONLY (e.g. EN-77)
- in_stock: number only
- failure_days: number
- schedule_by_days: number, less than failure_days
- priority: EXACTLY "URGENT" or "NORMAL" - no notes
- summary: one-line request; put any escalation note HERE
```

1. Turn **ON** the toggle **"Require Specific Output Format"** (bottom of Agent C's Parameters). An **Output Parser** slot appears under the node.
2. Under the **Output Parser** slot → **+** → **Structured Output Parser** → Schema Type = **Generate From JSON Example** → paste:

```
{
  "machine": "IM-500 (Bay 3)",
  "component": "Servo Encoder EN-77",
  "part": "EN-77",
  "in_stock": 0,
  "failure_days": 6,
  "schedule_by_days": 6,
  "priority": "URGENT",
  "summary": "Raise URGENT maintenance; EN-77 out of stock, 21-day lead exceeds 6-day failure - escalate."
}
```

> The "all properties will be required" note is fine — Agent C always fills them.

1. Wire: **Agent B** → **Agent C**.

Step 5 — The Google Sheet + the Append Row node

5a. Create the sheet (`sheets.google.com`): blank spreadsheet → name it `IM500_Maintenance_Log`. Row 1 headers (A1–H1), one per cell:

```
Timestamp | Machine | Failing Component | Part No | In Stock | Failure In (days) | Schedule By (days) | Priority
```

5b. Add the node in n8n:

1. From **Agent C**, click **+** → search **Google Sheets** → add it.
2. **Credential** → **Sign in with Google** → allow access.
3. Configure:
   - **Operation** = **Append Row**
   - **Document** = From list → `IM500_Maintenance_Log`
   - **Sheet** = From list → `Sheet1`
   - **Mapping Column Mode** = **Map Each Column Manually**
4. In **Values to Send**, click each column, toggle it to **Expression**, and enter:

```
Timestamp          {{ $now }}
Machine            {{ $json.output.machine }}
Failing Component  {{ $json.output.component }}
Part No            {{ $json.output.part }}
In Stock           {{ $json.output.in_stock }}
Failure In (days)  {{ $json.output.failure_days }}
Schedule By (days) {{ $json.output.schedule_by_days }}
Priority           {{ $json.output.priority }}
```

1. Wire: **Agent C** → **Google Sheets**.

> If a value previews as blank, open Agent C's Output tab and confirm the fields sit under `output`. If they're at the top level, drop `.output` (e.g. `{{ $json.machine }}`).

Step 6 — Test the whole flow

Save → **Open Chat** → run:

```
Servo position error is now 140 counts and rising
```

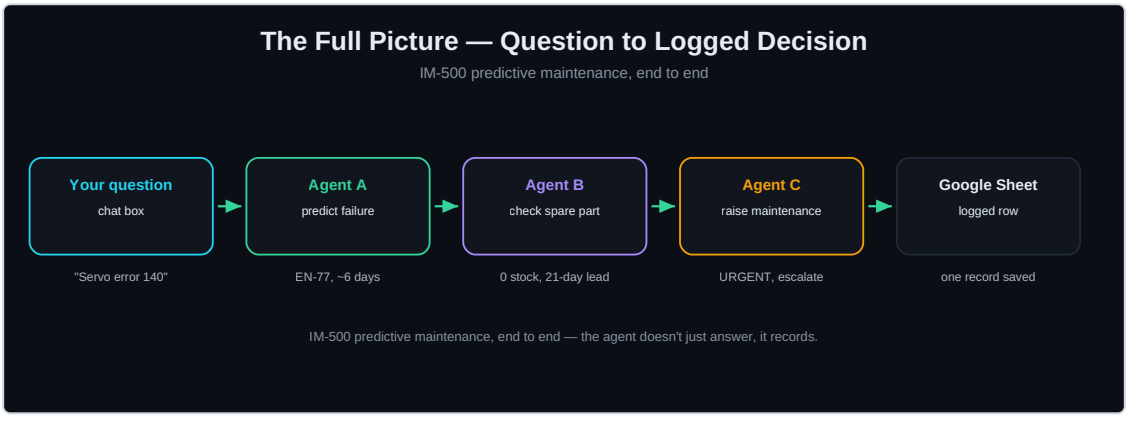
Expected: - **Agent A** → Servo Encoder EN-77, ~6 days (Critical) - **Agent B** → EN-77, 0 in stock, 21-day lead, URGENT - **Agent C** → structured maintenance request, schedule within 6 days, **priority URGENT**, escalate - **Google Sheet** → a new row appended

Contrast run (proves it's reasoning, not scripted):

```
Hydraulic pump vibration has been 3.9 mm/s for the last 4 shifts
```

→ pump bearing, BR-3105 in stock, **NORMAL**, schedule within 7 days, and a NORMAL row in the sheet.

The finished flow



End to end

`Chat → Agent A → Agent B → Agent C → Google Sheets` — one canvas, built once, no copy-paste.

Gotchas (quick)

- Only **Agent A** uses “Take from previous node automatically”; **B and C** use “Define below” + an Expression. The error `expected chatInput` = a downstream agent is still on auto.
- Two hops back → `.first()`, never `.item`.
- Each agent needs its **own** Chat Model sub-node; they can **share the same credential**.
- Node names must match exactly: `Agent A`, `Agent B` (the expressions reference them by name).
- Keep Agent C’s `priority` (URGENT/NORMAL) and `component` (from the fixed list) clean, or the sheet columns get messy.
- Stable model, not preview. On a 429, switch each agent’s model to a different stable one — quota is per-model.